

Labo 01 — Labo pratique : observer l'isolation de ses propres yeux

Objectif : vérifier expérimentalement chaque affirmation de la théorie. À la fin, vous aurez vu qu'un conteneur est un processus de votre machine WSL, et que le `root` d'un conteneur rootless, c'est vous.

Prérequis — Windows 10/11 avec WSL 2 et une distribution Ubuntu (22.04 ou plus récent). Aucun fichier n'est nécessaire pour ce labo. Les sorties montrées ont été produites avec Podman 5.8 ; à partir de Podman 4.9 les commandes sont identiques, seuls quelques détails d'affichage varient.

Étape 0 — Préparer WSL et installer Podman

Dans un terminal **PowerShell** (Windows) :

```
wsl --version          # WSL version 2.x attendu
wsl --list --verbose    # votre Ubuntu doit être en VERSION 2
```

Puis dans le terminal **Ubuntu** :

```
cat /etc/wsl.conf
```

Observez s'il contient `[boot]` puis `systemd=true`. Si ce n'est pas le cas, ajoutez-le :

```
printf '[boot]\nsystemd=true\n' | sudo tee /etc/wsl.conf
```

puis, depuis PowerShell, `wsl --shutdown` et rouvrez Ubuntu. Installez ensuite Podman :

```
sudo apt update && sudo apt install -y podman
podman --version
```

→ WINDOWS / WSL

WSL 2 est une VM Hyper-V minuscule qui démarre en une seconde et partage la RAM avec Windows. Par défaut, elle n'a **pas** de `systemd` : c'est un choix historique de Microsoft, et c'est précisément `systemd` qui délègue à votre utilisateur le droit de créer des *cgroups*. Sans lui, `podman run --memory` et `podman stats` ne fonctionnent pas en rootless. D'où l'étape ci-dessus. Vous n'avez pas besoin de Docker Desktop, ni de Podman Desktop : Podman est ici un simple paquet Ubuntu. (Si vous utilisez malgré tout Podman Desktop, `podman machine` crée sa propre distribution WSL et les commandes de ce labo sont identiques.)

Étape 1 — Identifier le moteur

```
podman version
```

Observez un seul bloc, `Client: Podman Engine`, avec `Version: 5.x.x` et `OS/Arch: linux/amd64`. Pas de bloc `Server`.

```
podman info | head -n 40
podman info --format 'rootless={{.Host.Security.Rootless}} cgroups={{.Host.CgroupManager}}
réseau={{.Host.NetworkBackend}} runtime={{.Host.OCIRuntime.Name}}'
```

Observez `rootless=true cgroups=systemd réseau=netavark runtime=crun`, et dans la sortie longue les lignes `kernel: 6.6.87.2-microsoft-standard-WSL2`, `idMappings:` (nous y revenons à l'étape 5) et `graphRoot: /home/<vous>/.local/share/containers/storage`.

Explication. Chez Docker, `version` interroge deux moitiés — client et daemon — et `info` décrit le daemon. Chez Podman il n'y a qu'un programme : `podman info` décrit ce que **votre utilisateur** peut faire. Le `graphRoot` dans votre `home` le confirme : les images ne sont pas dans `/var/lib`, elles vous appartiennent.

Étape 2 — Le premier conteneur, et où il est passé

```
podman run alpine echo "bonjour depuis le conteneur"
```

Observez d'abord `Resolved "alpine" as an alias (/etc/containers/registries.conf.d/000-shortnames.conf)`, puis `Trying to pull docker.io/library/alpine:latest...`, des lignes `Copying blob`, `Writing manifest`, le message affiché... puis le retour immédiat au prompt.

PODMAN

Docker complète silencieusement `alpine` en `docker.io/library/alpine`. Podman refuse de deviner : il consulte une liste d'alias connus (`alpine`, `nginx`, `debian`, `node`, `postgres` ...) et, pour un nom inconnu, il vous **demande** sur quel registry chercher — ou échoue si aucun terminal n'est disponible. C'est pour cela que les Dockerfiles et scripts d'entreprise écrivent toujours le nom complet : `docker.io/library/eclipse-temurin:21-jre`. Prenez l'habitude dès maintenant.

```
podman ps
podman ps -a
```

Observez que `podman ps` ne montre **rien**, mais que `podman ps -a` montre le conteneur, avec un nom aléatoire (`trusting_sanderson` ...), l'image sous son nom complet `docker.io/library/alpine:latest`, et le statut `Exited (0)`.

```
podman run --rm alpine echo "celui-ci ne laissera pas de trace"
podman ps -a
```

Observez qu'aucun nouveau conteneur n'apparaît : `--rm` supprime le conteneur à sa sortie.

Explication. Un conteneur vit exactement le temps de son processus principal. `echo` a écrit une ligne puis s'est terminé : le conteneur est mort avec lui, mais il n'est pas supprimé — il reste comme un cadavre inspectable. `podman ps` ne liste que les conteneurs en cours d'exécution.

Étape 3 — Le noyau est celui de l'hôte (et l'hôte, c'est WSL)

```
uname -r
podman run --rm alpine uname -r
podman run --rm debian uname -r
```

Observez que les **trois** commandes affichent la même valeur, `6.6.87.2-microsoft-standard-WSL2` par exemple — alors qu'Ubuntu, Alpine et Debian sont trois systèmes différents.

```
podman run --rm alpine cat /etc/os-release | head -n 2
podman run --rm debian cat /etc/os-release | head -n 2
```

Observez cette fois deux résultats différents : `Alpine Linux` et `Debian GNU/Linux`.

Explication. La preuve est faite : l'image apporte le *userland* (fichiers, binaires, bibliothèques), le noyau vient de l'hôte et n'est jamais dupliqué. Et cet hôte n'est pas Windows : le suffixe `microsoft-standard-WSL2` est la signature du noyau Linux que Microsoft compile pour WSL. Vos conteneurs tournent dans cette VM.

→ LINUX

`/etc/os-release` est un simple fichier texte que chaque distribution installe pour se présenter. `uname -r`, lui, est un **appel système** : la réponse vient du noyau. C'est pour cela que le premier varie d'un conteneur à l'autre et pas le second.

Étape 4 — Voir le processus depuis les deux côtés

Lancez un conteneur qui dure :

```
podman run -d --name veilleur alpine sleep 600
podman ps
```

Observez le statut `Up`, le nom `veilleur`, et la commande `sleep 600`.

Vue **depuis l'intérieur** :

```
podman exec veilleur ps -o pid,ppid,comm
```

Observez une liste minuscule : `sleep` porte le **PID 1**, et votre `ps` le PID 2.

Vue **depuis l'hôte** :

```
ps -ef | grep "[s]leep 600"
podman inspect --format '{{.State.Pid}}' veilleur
```

Observez que le même processus existe sur l'hôte, appartenant à **votre utilisateur**, avec un PID ordinaire (`1854` par exemple), et que `podman inspect` vous donne précisément ce PID.

```
podman top veilleur
```

Observez `USER root`, `PID 1`, `COMMAND sleep 600` : c'est la vue « conteneur » du même processus, reconstruite par Podman.

Explication. Un seul et même processus, deux numérotations. À l'intérieur, le namespace `pid` lui fait croire qu'il est le premier processus du système ; à l'extérieur, il n'est qu'un processus parmi des centaines, et il est à vous. C'est toute l'idée du conteneur.

Vérifiez qu'on peut retirer cette isolation :

```
podman run --rm --pid=host alpine ps -o pid,comm | head -n 8
```

Observez les processus de **votre WSL** (`init`, `systemd`, `conmon`...) listés depuis l'intérieur d'un conteneur.

Explication. L'isolation est une option, pas une propriété intrinsèque. C'est pourquoi `--pid=host` et `--privileged` sont interdits par défaut en production. Notez au passage `conmon` : c'est le petit superviseur que Podman laisse derrière chaque conteneur, puisqu'il n'y a pas de daemon pour le faire.

Étape 5 — Le root qui n'en est pas un (rootless)

```
podman exec veilleur id
```

Observez `uid=0(root) gid=0(root)` : dans le conteneur, `sleep` tourne en root.

```
podman top veilleur user,huser,pid,hpid,comm
```

Observez :

USER	HUSER	PID	HPID	COMMAND
root	1000	1	1854	sleep 600

`USER` est l'identité vue du conteneur, `HUSER` l'identité réelle sur l'hôte : `1000`, c'est vous (`id -u` pour vérifier).

```
podman unshare cat /proc/self/uid_map
```

Observez une table de correspondance du type :

0	1000	1
1	100000	65536

Explication. C'est le namespace `user` en action. Ligne 1 : l'UID `0` du conteneur **est** votre UID `1000`. Ligne 2 : les UID `1` à `65536` du conteneur sont projetés sur une plage d'UID « de réserve » (`100000` +, définie dans `/etc/subuid`) que personne d'autre n'utilise. Le « root » du conteneur n'a donc, sur l'hôte, que vos droits. Un conteneur compromis ne peut pas devenir root sur votre WSL : il n'y a rien à escalader.

→ SÉCURITÉ

Chez Docker, le daemon tourne en root et un `root` de conteneur est, sauf configuration spéciale, le vrai root de l'hôte. L'isolation repose alors uniquement sur les namespaces `pid` / `mnt` / `net` et sur les *capabilities* retirées. Podman rootless ajoute une couche que Docker n'a pas par défaut : même si tout le reste cède, l'attaquant est un utilisateur ordinaire.

Étape 6 — Image immuable, conteneur jetable

```
podman run -d --name c1 alpine sleep 600
podman run -d --name c2 alpine sleep 600
podman exec c1 sh -c 'echo "donnee de c1" > /marque.txt'
```

Vérifiez l'isolation des écritures :

```
podman exec c1 cat /marque.txt      # affiche : donnee de c1
podman exec c2 cat /marque.txt      # cat: can't open '/marque.txt': No such file or directory
```

Vérifiez que l'image, elle, n'a pas bougé :

```
podman run --rm alpine ls /marque.txt  # No such file or directory
```

Mesurez cette couche :

```
podman ps -s --format 'table {{.Names}}\t{{.Size}}'
```

Observez une taille du type `11.4kB (virtual 8.72MB)` : le `virtual` est la taille image + couche, la première valeur est ce que le conteneur consomme **en propre** — quelques kilo-octets de métadonnées, plus votre fichier.

Enfin, détruisez et recommencez :

```
podman rm -f -t 0 c1
podman run -d --name c1 alpine sleep 600
podman exec c1 ls /marque.txt      # No such file or directory
```

Explication. `podman rm` détruit le conteneur **et** sa couche d'écriture. Le nouveau `c1` repart de l'état exact de l'image. Toute donnée à conserver doit sortir du conteneur : c'est l'objet du labo 06.

PODMAN

Pourquoi `-t 0` ? `podman rm -f` commence par un arrêt poli (`SIGTERM`), attend **10 secondes**, puis tue. Docker, lui, tue immédiatement. Comme `sleep` ignore `SIGTERM` (labo 03), sans `-t 0` vous attendriez dix secondes en regardant un avertissement `StopSignal SIGTERM failed to stop container ... resorting to SIGKILL`. Ce n'est pas un bug : c'est Podman qui vous dit que votre application ne s'arrête pas proprement.

Étape 7 — Les cgroups, ou la limite de consommation

```
podman run -d --name limite --memory=128m --memory-swap=128m alpine sleep 600
podman stats --no-stream limite
```

Observez la colonne `MEM USAGE / LIMIT` : `471kB / 134.2MB`, et non la RAM totale de votre machine.

Comparez avec un conteneur sans limite :

```
podman stats --no-stream veilleur
```

Observez que la limite affichée est la RAM totale... **de la VM WSL**, par exemple `7.7GB` sur un PC de 16 Go.

Explication. Sans `--memory`, un conteneur peut consommer toute la mémoire disponible. Le namespace ne protège de rien ici : c'est le cgroup qui plafonne. Si cette étape échoue avec `OCI runtime error: ... cgroup ...`, c'est que `systemd` n'est pas actif dans votre WSL (étape 0).

→ **WINDOWS / WSL**

WSL 2 ne voit par défaut que **50 % de la RAM** de Windows (et 8 Go au plus sur les anciennes versions). C'est réglable dans `%UserProfile%\wslconfig` (`[wsl2]` puis `memory=12GB`). Quand un conteneur « manque de mémoire » sur un poste Windows, la limite qui compte est souvent celle-là, pas celle du conteneur.

Étape 8 — `inspect`, la source de vérité

```
podman inspect veilleur | head -n 30
```

C'est verbeux : ciblez ce qui vous intéresse avec un *format Go*.

```
podman inspect --format '{{.State.Status}}' veilleur
podman inspect --format '{{.Config.Image}}' veilleur
podman inspect --format '{{json .Config.Cmd}}' veilleur
podman inspect --format '{{.NetworkSettings.IPAddress}}' veilleur
```

Observez respectivement `running`, `docker.io/library/alpine:latest`, `["sleep","600"]` ... et **une ligne vide** pour l'adresse IP.

```
podman exec veilleur ip -4 addr show eth0
```

Observez que le conteneur a pourtant une interface `eth0`, avec **la même adresse IP que votre WSL** (`172.2x.x.x`).

Explication. En rootless, un utilisateur ordinaire n'a pas le droit de créer un pont réseau. Podman utilise donc `pasta`, un traducteur en espace utilisateur qui *copie* l'adresse de l'hôte dans le conteneur ; il n'y a pas d'IP « de conteneur » à afficher. Ce sera un thème du labo 07. Retenez pour l'instant que la valeur vide n'est pas une erreur, et que `--network podman` vous donnerait un vrai pont avec une IP `10.88.0.x` :

```
podman run -d --network podman --name ponte alpine sleep 600
podman inspect --format '{{.NetworkSettings.Networks.podman.IPAddress}}' ponte
```

Observez `10.88.0.2`. Comparez avec les métadonnées de l'**image** :

```
podman image inspect --format '{{json .Config.Cmd}}' alpine
podman image inspect --format '{{.Architecture}}/{{.Os}}' alpine
```

Observez que l'image porte elle aussi une commande par défaut (`["/bin/sh"]`), que votre `sleep 600` a écrasée au `run`, et `amd64/linux`.

Explication. `podman inspect` fonctionne sur **tous** les objets (conteneur, image, volume, réseau) et donne l'état réel, sans mise en forme. Quand la documentation et la réalité divergent, `inspect` a

raison.

Étape 9 — La CLI, forme longue, forme courte... et `docker`

```
podman container ls -a
podman ps -a
podman image ls
podman images
```

Observez des sorties identiques deux à deux.

```
podman container ls --format 'table {{.Names}}\t{{.Image}}\t{{.Status}}'
```

Maintenant, faites-vous passer pour Docker :

```
alias docker=podman
docker ps
docker images
```

Observez que tout fonctionne. Pour rendre l'alias permanent : `echo 'alias docker=podman' >> ~/.bashrc`. Sur Ubuntu, le paquet `podman-docker` fait la même chose (il fournit un binaire `docker` qui appelle `podman`).

Explication. `--format` accepte un gabarit Go et rend les sorties exploitables en script — bien plus fiable que de découper le tableau par défaut avec `awk`. Quant à l'alias : la compatibilité de la CLI est une promesse de Podman, et c'est ce qui vous permet de suivre n'importe quel tutoriel Docker.

Nettoyage

```
podman rm -f -t 0 veilleur c1 c2 limite ponté
podman ps -a
```

Il reste le conteneur `Exited` de l'étape 2. Supprimez-le nommément :

```
podman ps -a --filter status=exited --format '{{.Names}}'
podman rm <nom>
```

Et si vous voulez récupérer l'espace de l'image Debian, qui ne resservira pas :

```
podman images
podman rmi debian          # on garde alpine pour les labos suivants
```

PIÈGE

vous croiserez partout `podman container prune`, `podman image prune -a` et `podman system prune -a`. Ces commandes ne suppriment pas « ce que vous venez de faire » mais **tout ce qui n'est pas utilisé** : les images et conteneurs de vos autres projets partent avec. Supprimez toujours nommément. Nous verrons `prune` proprement au labo 10.

Ce que vous devez pouvoir affirmer maintenant

- Le noyau affiché dans un conteneur est celui de l'hôte — ici, celui de WSL 2.
- Le processus d'un conteneur existe dans le `ps` de l'hôte, sous **votre** utilisateur — vous l'avez vu, avec son PID.
- Le `root` d'un conteneur rootless est une projection de votre UID : `podman unshare cat /proc/self/uid_map` le prouve.
- Une écriture dans un conteneur n'atteint ni l'image, ni les autres conteneurs.
- `podman rm` détruit les données ; `podman stop` non. `podman rm -f` attend 10 s sans `-t 0`.
- Sans `--memory`, la seule limite est la RAM de la VM WSL.
- `podman inspect --format` est votre premier réflexe de diagnostic — et une IP vide en rootless est normale.